AI Use Declaration Form

Course: Introduction to Artificial Intelligence
Lab Title: Lab 1 Part A — Uninformed Search Algorithms
Student Name: ___ Aluong Leek Alier Leek____
Student ID: ____78152028____
GitHub Repository Link: _________ ________________________________
Date Submitted: _____20/06/2026____

1. AI Use Summary

Question	**Student Response**
Did you use any AI tool for this lab?	**yes**
Estimated percentage of the work influenced by AI	20 %
Did you attach evidence of AI use where applicable?	**yes**

2. Details of AI Use

Complete the table below for each AI tool used. Add more rows where necessary.

Name of AI Tool Used	**Purpose for Using the Tool**	**Prompt or Instruction Given to the Tool**	**Part(s) of the Work Influenced by the Tool**	**How I Verified, Edited, or Improved the AI Output**
Claude	To understand uninformed search algorithms and get guidance on debugging code.	Can you explain Depth-Limited Search in plain steps, like a teacher? I want to understand: - how it checks for the goal, - how it handles the depth limit, - how it expands children safely without cycles, - and how success, cutoff, and failure are decided.	N/A	I only used AI to understand the ideas. I checked the explanations with my class notes and made sure I understood them myself.

3. Attachment of AI Output Evidence

Where required, attach evidence of AI use. Tick all that apply. Add more rows where necessary.

Evidence Type	Attached? (Yes, No, N/A)	File Name
AI-generated draft	yes	N/A
Prompt history	yes	
Screenshot of AI interaction	yes	
Exported AI conversation	no	
AI-generated code snippet	yes	
Revised version showing student input	yes	

4. Student Declaration

I declare that:

Declaration Statement	Response (Yes/No)
The submitted work is my own work.	yes
Any use of AI tools has been clearly declared in this form.	YES
The prompts, instructions, and AI-generated outputs have been disclosed where applicable.	YES
I reviewed, tested, edited, and improved any AI-generated content before submission.	YES
My AI usage does not exceed 25% of the entire work.	yes
I understand that undeclared or excessive AI use may be treated as academic misconduct.	yes

Student Signature: __ leek____

Date: ________20/06/2026__________________________________ explain the concept of BFS, DFS, IDFS LIKE YOU R EXPLAINING IT TO A CHILD

Alright Aluong, let's make these search algorithms sound like a fun story for a child:

🟦 Breadth-First Search (BFS)

Imagine you are looking for candy in a big house.

- First, you check **all the rooms on the first floor.**
- If you don't find candy, you go to the **second floor** and check all those rooms.
- You keep going floor by floor.
  👉 BFS = explore level by level, closest first.

🟩 Depth-First Search (DFS)

This one is like playing a game with a **ladder that grows taller each time**.

- First, you climb only **one step** and look for candy.
- If not found, you climb **two steps** and look again.
- Then three steps, then four... until you find the candy.
  👉 IDFS = repeat DFS with bigger depth limits, so you don't miss candy that's far away.

✨ In short:

- **BFS** = check all neighbors first (wide search).
- **DFS** = go down one path fully (deep search).
- **IDFS** = repeat DFS with increasing limits (like taller ladders each time).

SAMPLE CODE TO DIFFERENTIATE THE THREE

Here's a **simple sample code** that shows the difference between BFS, DFS, and IDFS. I'll keep it short and clear so you can see the concept without too much detail:

```
from collections import deque


# Example graph as adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}


# Breadth-First Search (BFS)
def bfs(start, goal):
    queue = deque([start])
    visited = set()
    while queue:
        node = queue.popleft()
        if node == goal:
            return "Found with BFS"
        if node not in visited:
            visited.add(node)
            queue.extend(graph[node])
```

```
    return "Not found"

# Depth-First Search (DFS)
def dfs(start, goal, visited=None):
    if visited is None:
        visited = set()
    if start == goal:
        return "Found with DFS"
    visited.add(start)
    for child in graph[start]:
        if child not in visited:
            result = dfs(child, goal, visited)
            if result:
                return result
    return None

# Iterative Deepening DFS (IDFS)
def dls(node, goal, limit, visited):
    if node == goal:
        return True
    if limit <= 0:
        return False
    visited.add(node)
    for child in graph[node]:
        if child not in visited:
            if dls(child, goal, limit-1, visited):
                return True
```

```python
    return False

def idfs(start, goal, max_depth):
    for depth in range(max_depth+1):
        visited = set()
        if dls(start, goal, depth, visited):
            return f"Found with IDFS at depth {depth}"
    return "Not found"

# Example runs
print(bfs('A', 'F'))
print(dfs('A', 'F'))
print(idfs('A', 'F', 3))
```